

Dart Language Features Guide

This guide organizes key Dart language features, starting with classes of data (primitive and reference types) and incorporating operators, followed by other essential features. Each category lists relevant Dart features (keywords, constructs, or functions), with a table detailing the feature, its uses, an example, and practical applications. The focus is on core Dart features for general-purpose programming, excluding Flutter-specific elements, based on Dart 3.x documentation for clarity and relevance.

Classes of Data: Primitive Types

Primitive types in Dart are basic data types that hold simple values directly, not references to objects. Dart is object-oriented, so even primitives are objects under the hood, but they behave as simple values.

Features: `int`, `double`, `bool`, `String`, `null`

Feature	Uses	Example	Practical Uses
<code>int</code>	Integer values (64-bit on most platforms).	<code>int score = 100;</code>	Counters, indices, or IDs in algorithms.
<code>double</code>	Floating-point numbers.	<code>double price = 19.99;</code>	Financial calculations or measurements.
<code>bool</code>	Boolean values (<code>true</code> / <code>false</code>).	<code>bool isActive = true;</code>	Conditional flags for logic or validation.
<code>String</code>	Sequence of UTF-16 characters.	<code>String name = "Dart";</code>	Text processing for logs or user input.
<code>null</code>	Represents absence of value (with null safety).	<code>String? value = null;</code>	Optional or uninitialized variables.

Classes of Data: Reference Types

Reference types are objects that store references to data in memory, including collections, user-defined classes, and other complex types. These are mutable or immutable based on their implementation.

Features: `List`, `Map`, `Set`, `Object?`, `dynamic`, user-defined class

Feature	Uses	Example	Practical Uses
<code>List</code>	Ordered collection of items.	<code>List<int> numbers = [1, 2, 3];</code>	Storing sequences like task lists.
<code>Map</code>	Key-value pair collection.	<code>Map<String, int> scores = {'Alice': 90};</code>	Storing structured data like configs.
<code>Set</code>	Unordered collection of unique items.	<code>Set<String> tags = {'new', 'sale'};</code>	Deduplicating data like categories.

Feature	Uses	Example	Practical Uses
Object?	Base type for all objects, nullable.	<code>Object? data = 42;</code>	Generic type for flexible APIs.
dynamic	Bypasses static type checking.	<code>dynamic value = "text";</code>	Handling JSON or mixed-type data.
user-defined class	Custom object blueprint.	<code>class Person { String name; Person(this.name); }</code>	Modeling entities like users or products.

Operators

Dart operators for arithmetic, comparison, logical, and other operations.

Features: +, -, *, /, %, ==, !=, <, >, <=, >=, &&, ||, !, ??, is, as, ~/ (integer division), ++, --

Feature	Uses	Example	Practical Uses
+	Addition or string concatenation.	<code>int sum = 5 + 3; String text = "Hello" + " World";</code>	Calculating totals or building strings.
-	Subtraction.	<code>int diff = 5 - 3;</code>	Computing differences like balances.
*	Multiplication.	<code>int product = 5 * 3;</code>	Scaling values in computations.
/	Division (returns double).	<code>double result = 10 / 3;</code>	Calculating averages or ratios.
%	Modulus (remainder).	<code>int remainder = 10 % 3;</code>	Checking divisibility or cycling.
==	Equality check.	<code>if (x == y)</code>	Comparing values like user IDs.
!=	Inequality check.	<code>if (x != y)</code>	Ensuring distinct values.
<, >, <=, >=	Comparison operators.	<code>if (x < 10)</code>	Filtering data like age or price.
&&	Logical AND.	<code>if (x > 0 && y > 0)</code>	Combining conditions in validation.
			Logical OR.
!	Logical NOT.	<code>if (!isValid)</code>	Inverting boolean conditions.
??	Null-coalescing (default if null).	<code>String display = name ?? "Unknown";</code>	Providing defaults for null values.

Feature	Uses	Example	Practical Uses
is	Type checking.	<code>if (value is String)</code>	Validating types in mixed data.
as	Type casting.	<code>var str = value as String;</code>	Converting types safely.
~/	Integer division (returns int).	<code>int quotient = 10 ~/ 3;</code>	Floor division for counters.
++	Increments by 1.	<code>x++;</code>	Loop counters or increments.
--	Decrements by 1.	<code>x--;</code>	Decreasing counters in loops.

Control Flow

Features for controlling program execution.

Features: if, else, switch, for, for-in, while, do-while, break, continue, assert

Feature	Uses	Example	Practical Uses
if	Conditional execution.	<code>if (score > 50) { print("Pass"); }</code>	Validating conditions like input checks.
else	Alternative execution path.	<code>if (isEmpty) {} else { print("Not empty"); }</code>	Handling default cases.
switch	Multi-way branching.	<code>switch (grade) { case 'A': print("Excellent"); break; }</code>	Processing multiple states.
for	Loops with counter.	<code>for (var i = 0; i < 5; i++) { print(i); }</code>	Iterating over arrays or ranges.
for-in	Iterates over iterables.	<code>for (var item in list) { print(item); }</code>	Processing list or set elements.
while	Loops while condition is true.	<code>while (retries > 0) { retries--; }</code>	Polling or retry logic.
do-while	Loops at least once.	<code>do { print(x--); } while (x > 0);</code>	Ensuring one iteration for tasks.
break	Exits loop or switch.	<code>for (var i = 0; i < 5; i++) { if (i == 3) break; }</code>	Early loop termination.
continue	Skips current loop iteration.	<code>for (var i = 0; i < 5; i++) { if (i == 2) continue; }</code>	Skipping invalid items.
assert	Checks condition during debugging.	<code>assert(x >= 0, "Must be non-negative");</code>	Validating invariants in code.

Functions

Features for defining reusable code blocks.

Features: function declaration, arrow function, named parameters, optional parameters, lambda (closure), Function type

Feature	Uses	Example	Practical Uses
function declaration	Defines named function.	<code>int add(int a, int b) { return a + b; }</code>	Reusable logic like calculations.
arrow function	Concise single-expression function.	<code>int square(int x) => x * x;</code>	Short utility functions.
named parameters	Parameters passed by name.	<code>void greet({required String name}) => print("Hi, \$name");</code>	Clear API for function calls.
optional parameters	Parameters with default/null values.	<code>void log([String? msg]) => print(msg ?? "Log");</code>	Flexible function inputs.
lambda (closure)	Anonymous function capturing scope.	<code>var add = (int x, int y) => x + y;</code>	Callbacks in async operations.
Function type	Type for function references.	<code>Function callback = (int x) => print(x);</code>	Passing functions as arguments.

Collections and Iterables

Features for working with collections.

Features: List.add, List.remove, Map.putIfAbsent, Set.add, List.map, List.where, fold, forEach

Feature	Uses	Example	Practical Uses
List.add	Adds item to list.	<code>numbers.add(4);</code>	Appending data to dynamic lists.
List.remove	Removes first matching item.	<code>numbers.remove(2);</code>	Deleting items from lists.
Map.putIfAbsent	Adds key-value pair if key absent.	<code>scores.putIfAbsent("Bob", () => 80);</code>	Default values in configs.
Set.add	Adds item to set.	<code>tags.add("new");</code>	Adding unique elements.

Feature	Uses	Example	Practical Uses
List.map	Transforms elements to new iterable.	<code>numbers.map((n) => n * 2).toList();</code>	Transforming data for processing.
List.where	Filters elements by condition.	<code>numbers.where((n) => n > 0).toList();</code>	Filtering valid records.
fold	Reduces iterable to single value.	<code>numbers.fold(0, (sum, n) => sum + n);</code>	Summing values like totals.
forEach	Applies function to each element.	<code>numbers.forEach((n) => print(n));</code>	Logging or processing items.

Object-Oriented Programming

Features for defining and working with classes and objects.

Features: class, constructor, named constructor, this, extends, implements, abstract, factory, getter/setter

Feature	Uses	Example	Practical Uses
class	Defines object blueprint.	<code>class Person { String name; }</code>	Modeling entities like users.
constructor	Initializes objects.	<code>Person(this.name);</code>	Setting up object state.
named constructor	Alternative constructor logic.	<code>Person.guest() : name = "Guest";</code>	Default or special instances.
this	Refers to current instance.	<code>this.name = name;</code>	Assigning constructor arguments.
extends	Inherits from parent class.	<code>class Employee extends Person {}</code>	Reusing code in hierarchies.
implements	Implements interface methods.	<code>class Logger implements Printable { void print() {} }</code>	Ensuring method contracts.
abstract	Defines non-instantiable class.	<code>abstract class Shape { void draw(); }</code>	Base classes for models.
factory	Returns instance	<code>factory Person.fromJson(Map json) { return Person(json['name']); }</code>	Parsing data to

Feature	Uses	Example	Practical Uses
	from constructor.		objects.
getter/setter	Controls property access.	<code>String get fullName => name;</code>	Computed properties in models.

Null Safety

Features for safe handling of null values.

Features: `?`, `!`, `??`, `late`, `required`

Feature	Uses	Example	Practical Uses
<code>?</code>	Declares nullable type.	<code>String? name;</code>	Optional data fields.
<code>!</code>	Asserts non-null value.	<code>String value = name!;</code>	Accessing nullable values safely.
<code>??</code>	Provides default for null.	<code>String display = name ?? "Unknown";</code>	Default values for missing data.
<code>late</code>	Delays initialization.	<code>late String title;</code>	Initializing variables at runtime.
<code>required</code>	Marks named parameter as mandatory.	<code>void setData({required String id}) {}</code>	Enforcing required arguments.

Asynchrony

Features for handling asynchronous operations.

Features: `async`, `await`, `Future`, `Stream`, `async*`, `yield`

Feature	Uses	Example	Practical Uses
<code>async</code>	Marks function as asynchronous.	<code>Future<int> fetch() async { return 42; }</code>	Async data fetching.
<code>await</code>	Waits for Future completion.	<code>int result = await fetch();</code>	Sequential async processing.
<code>Future</code>	Represents future value.	<code>Future.delayed(Duration(seconds: 1), () => 42);</code>	Delayed tasks or API calls.
<code>Stream</code>	Sequence of async events.	<code>Stream<int> count() async* { yield 1; yield 2; }</code>	Real-time data like logs.
<code>async*</code>	Defines stream generator.	<code>Stream<int> numbers() async* { yield 1; }</code>	Streaming data sequences.

Feature	Uses	Example	Practical Uses
yield	Emits value in stream.	<code>yield 3;</code>	Producing stream values.

Standard Libraries

Common Dart libraries for extended functionality.

Features: dart:core, dart:math, dart:io, dart:convert, dart:async

Feature	Uses	Example	Practical Uses
dart:core	Core types and functions (List, Map).	<code>List<int> nums = [1, 2];</code>	Basic data structures.
dart:math	Mathematical functions and constants.	<code>import 'dart:math'; double root = sqrt(16);</code>	Calculations in algorithms.
dart:io	File and network I/O.	<code>import 'dart:io'; File('data.txt').readAsString();</code>	Reading/writing files.
dart:convert	Data encoding/decoding (JSON, UTF-8).	<code>import 'dart:convert'; jsonDecode('{ "key": "value" });</code>	Parsing API responses.
dart:async	Async programming utilities.	<code>import 'dart:async'; Timer(Duration(seconds: 1), () {});</code>	Scheduling tasks or timeouts.

Exception Handling

Features for managing errors and exceptions.

Features: try, catch, on, finally, throw

Feature	Uses	Example	Practical Uses
try	Monitors block for exceptions.	<code>try { int x = 1 ~/ 0; }</code>	Wrapping risky operations.
catch	Handles exceptions with stack trace.	<code>catch (e, s) { print(e); }</code>	Logging errors.
on	Handles specific exception types.	<code>on FormatException catch (e) { print("Invalid format"); }</code>	Type-specific error handling.
finally	Runs regardless of exception.	<code>finally { print("Cleanup"); }</code>	Releasing resources.

Feature	Uses	Example	Practical Uses
throw	Throws an exception.	<code>throw Exception("Invalid input");</code>	Custom error signaling.